Ninsho: An Authentication and Session Management SDK for Node.js

Project Report

Submitted in fulfillment of the requirements

for the degree of

Bachelor of Technology

in

Information Technology

by

Jain Darshan Arvind (24101B2002)
Jadhav Yash Gangadhar (24101B2003)
Padhi Chetna Shivshankar (24101B2005)
Sharma Chirag Pradeep (24101C2007)

Project Guide

Dr. Varsha Bhosale



Department of Information Technology
Vidyalankar Institute of Technology
Wadala (E), Mumbai – 400037
(Affiliated to University of Mumbai)

2025-2026

Certificate

This is to certify that the project entitled

“Ninsho: An Authentication and Session Management SDK for Node.js”

is a bonafide work done by

Jain Darshan Arvind (24101B2002)
Jadhav Yash Gangadhar (24101B2003)
Padhi Chetna Shivshankar (24101B2005)
Sharma Chirag Pradeep (24101C2007)

and is submitted in fulfillment of the requirement for the degree of

Bachelor of Technology

in Information Technology

to the

Vidyalankar Institute of Technology, Wadala, Mumbai.

Project Guide	Head of Department	Principal

Dissertation Approval for B.Tech.

This is to certify that the dissertation entitled “Ninsho: An Authentication and Session Management SDK for Node.js” is a bonafide work done by [Student Names and Roll Numbers] under the supervision of [Dr. / Prof. Supervisor Name]. This dissertation has been approved for the award of the B.Tech. degree in Information Technology, Vidyalankar Institute of Technology, Wadala, Mumbai.

Examiners:

1. _____

2. _____

Date: _____

Place: _____

Declaration

I declare that this written submission represents my ideas in my own words and where others' ideas or words have been included, I have adequately cited and referenced the original sources. I also declare that I have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in my submission. I understand that any violation of the above will be cause for disciplinary action by the Institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

1. _____

2. _____

3. _____

4. _____

(Roll No., Name and Signature of Students)

Date: _____

Place: _____

Acknowledgement

We express our sincere gratitude to our project guide, Dr. Varsha Bhosale, Department of Information Technology, for guidance, encouragement and critical feedback throughout this project.

We thank the Head of the Department, Dr. Vidya Chitre, the Principal, Dr. Sangeeta Joshi, and the faculty and laboratory staff of Vidyalankar Institute of Technology for the facilities and support provided. We also thank our families and friends for their constant support.

Jain Darshan Arvind (24101B2002)
Jadhav Yash Gangadhar (24101B2003)
Padhi Chetna Shivshankar (24101B2005)
Sharma Chirag Pradeep (24101C2007)

Abstract

Web applications often assemble authentication from loosely coupled token libraries, which leaves gaps in revocation, refresh-token theft handling and replay of stolen bearer credentials. This project presents Ninsho, an open-source authentication and session-management SDK for Node.js, and evaluates it against its own documented security properties. Ninsho issues opaque access tokens verified by a single server-side lookup, rotates single-use refresh tokens with reuse detection, optionally issues PASETO v4.public tokens, binds tokens to a browser-held key using DPoP proof-of-possession with replay protection, provides ownership and role guards, and includes an optional WebAuthn verifier. Session state is held in an injected store, with Redis as the production implementation. The evaluation combines more than two thousand automated tests, recorded as passing without skips in the project's own runs against a dedicated Redis 7 instance, a continuous-integration run in which all eight jobs, including Chromium, Firefox and WebKit regressions, succeeded, twenty-five recorded manual attack entries, Redis restart, outage and data-loss scenarios, and SecureShop, a demonstration application that contrasts a deliberately limited stateless JWT baseline with Ninsho. The results support the documented behaviours under the tested conditions; they do not prove security. Ninsho has not undergone an independent security audit, and revoked PASETO tokens can become valid again if Redis state is lost. A fresh local re-run on 21 October 2026 passed 2,151 automated tests with real Redis; a model-assisted code review (not an independent audit) found and fixed one PASETO revocation defect that occurred only under a non-default clock tolerance, and a single-machine benchmark compared Ninsho with a JWT baseline that performs an equivalent Redis revocation check.

Keywords: Authentication, Session Management, Refresh-Token Rotation, DPoP, PASETO, WebAuthn, Redis

Contents

Abstract v

List of Figures ix

List of Tables x

Abbreviations xi

Chapter 1 Introduction 1
- 1.1 Background 1
- 1.2 Motivation 2
- 1.3 Problem Statement 2
- 1.4 Objectives 3
- 1.5 Scope of the Work 3
- 1.6 Organization of the Report 4

Chapter 2 Literature Survey 5
- 2.1 Review of Existing Work 5
  - 2.1.1 Authentication schemes and web sessions 5
  - 2.1.2 OAuth, token theft and refresh-token security 5
  - 2.1.3 JWT security and implementation quality 6
  - 2.1.4 Proof-of-possession and sender-constrained tokens 6
  - 2.1.5 Passkeys and WebAuthn 6
- 2.2 Summary of Literature 7
- 2.3 Research Gaps 7

Chapter 3 Proposed Methodology 9
- 3.1 System Architecture 9
  - 3.1.1 Token engines and the one-read verification path 10
  - 3.1.2 Session manager, refresh tokens and the store 11
  - 3.1.3 Authorization guards and framework integration 11
  - 3.1.4 Browser client and cross-tab invalidation 12
  - 3.1.5 WebAuthn verification 12
- 3.2 Test Data and Experimental Inputs 13
- 3.3 Algorithm Description 13
  - 3.3.1 Session creation and request verification 13
  - 3.3.2 Refresh-token rotation and reuse detection 14
  - 3.3.3 Revocation 15
  - 3.3.4 DPoP proof verification 15
  - 3.3.5 Evaluation Metrics 16
- 3.4 Tools and Technologies Used 16

Chapter 4 Results and Discussion 18
- 4.1 Experimental Setup 18
- 4.2 Results 19
  - 4.2.1 Automated test suite and continuous integration 19

4.2.2 Security behaviours and their test evidence 20
4.2.3 Manual attack procedures 21
4.2.4 Browser evidence 22
4.2.5 Redis persistence, outage and data loss 23
4.2.6 SecureShop demonstration 23
4.2.7 Historical benchmark measurements 25
4.3 Comparison with Existing Methods 26
4.4 Limitations and Threats to Validity 27
Chapter 5 Conclusion and Future Scope 28
5.1 Conclusion 28
5.2 Future Scope 29
Chapter 6 List of Publications 31
6.1 List of Published Papers 31
6.2 Details of Participation in Project Competitions 31
References 32

List of Figures

Figure 3.1: Block diagram of the Ninsho architecture....9

Figure 3.2: Opaque token verification: one store read per request....10

Figure 3.3: Cross-tab invalidation after a confirmed logout....12

Figure 3.4: Refresh-token states, grace window and reuse detection....14

Figure 3.5: DPoP proof validation pipeline....15

Figure 3.6: Evidence sources used in this report....17

Figure 4.1: Tests per workspace in the recorded run (project-recorded)....20

Figure 4.2: SecureShop, Comparison A: replay of a copied JWT after client logout [insert screenshot]....24

Figure 4.3: SecureShop, Comparison C: wrong-key and replayed-proof requests [insert screenshot]....24

Figure 4.4: Historical verification throughput, release 0.1.0 (not for the current commit)....25

List of Tables

Table 2.1: Summary of literature survey 7

Table 3.1: Packages in the Ninsho monorepo (release 0.1.1) 10

Table 3.2: Opaque and PASETO v4.public strategies compared (from repository documentation) 11

Table 3.3: Test data and experimental inputs 13

Table 4.1: Evidence categories and their status 18

Table 4.2: Experimental environments recorded in the repository 19

Table 4.3: Security behaviours and the test files that evidence them 20

Table 4.4: Manual attack procedures grouped by theme (project-recorded, release 0.1.0) 22

Table 4.5: Browser regression results (project-recorded local run on Windows, Node 22.13.1, Playwright 1.64.1) 22

Table 4.6: Redis scenarios and recorded outcomes (project-recorded; Docker-managed tests) 23

Table 4.7: SecureShop documented outcomes by comparison 24

Table 4.8: Comparison of the SecureShop baseline with Ninsho configurations (design properties and documented outcomes) 26

Table 5.1: Objectives and the evidence supporting them 28

Abbreviations

AOF – Append-Only File
API – Application Programming Interface
BOLA – Broken Object Level Authorization
CBOR – Concise Binary Object Representation
CI – Continuous Integration
COSE – CBOR Object Signing and Encryption
CSPRNG – Cryptographically Secure Pseudo-Random Number Generator
DoS – Denial of Service
DPoP – Demonstrating Proof of Possession
FIDO – Fast IDentity Online
HTTP – Hypertext Transfer Protocol
IDOR – Insecure Direct Object Reference
JSON – JavaScript Object Notation
JWK – JSON Web Key
JWS – JSON Web Signature
JWT – JSON Web Token
PAE – Pre-Authentication Encoding
PASETO – Platform-Agnostic Security Tokens
RBAC – Role-Based Access Control
RDB – Redis Database snapshot
RFC – Request for Comments
RP – Relying Party
SDK – Software Development Kit
TTL – Time To Live
WebAuthn – Web Authentication
XSS – Cross-Site Scripting

Chapter 1
Introduction

Almost every networked application must answer two questions on every request: who is making it, and may that caller do what is being asked. The first is authentication and the second is authorization. Once a user has proved their identity, usually by a password or a passkey, the application issues a credential that stands in for that proof on later requests. How that credential is created, protected, renewed and withdrawn is called session management. It is a small part of an application's code and a large part of its attack surface.

This report describes the design and evaluation of **Ninsho**, an authentication and session-management software development kit (SDK) for Node.js. Ninsho does not verify passwords or federated sign-in; it takes over after identity is established and handles issuing, verifying, rotating and revoking session credentials, answering authorization questions, limiting request rates, and, optionally, verifying WebAuthn passkey ceremonies. The source code is public [1] and four packages are published on the npm registry under the scope `@ninshorg`.

1.1 Background

A *bearer token* is a credential that grants access to whoever presents it, without any proof that the presenter is the party to whom it was issued [2]. Bearer tokens are simple and widely deployed, but a token copied from a log file, a browser store or a network trace is as useful to an attacker as to its owner. Two designs dominate practice. In the first, the token is an opaque random string and the server looks it up in a store on each request; withdrawing a session means deleting a record. In the second, the token is self-contained and signed, as with a JSON Web Token (JWT) [3]; the server verifies the signature and reads the claims without consulting any store. The second design scales easily across servers, but a signed token remains acceptable until it expires unless the server adds state, such as a deny-list, a per-user session version or token introspection, to withdraw it earlier.

Short-lived access tokens are usually paired with longer-lived *refresh tokens*. Current OAuth 2.0 security guidance recommends that refresh tokens either be bound to the client

by sender-constraining or be rotated on use, so that presentation of a token that was already used signals theft [4]. *Sender-constrained* tokens can be achieved with Demonstrating Proof of Possession (DPoP), in which the client holds a private key and signs a small proof for every request, so that a stolen token without the key is not enough [5]. For passwordless sign-in, the W3C Web Authentication (WebAuthn) specification defines how a relying party registers and verifies public-key credentials held by authenticators [6].

Authentication alone does not protect user data. Broken object-level authorization, where a valid user requests another user's object by changing an identifier, tops the OWASP API Security list [7]. Libraries therefore need to make ownership checks easy to apply correctly. Implementation quality matters as well: a recent large-scale study of JWT libraries found previously unknown vulnerabilities in a substantial share of the implementations tested [8], and the IETF has published best-practice guidance for using JWTs safely [9].

1.2 Motivation

Developers who need sessions in a Node.js service typically combine a JWT library, a cookie handler, a rate limiter, a Redis client and hand-written middleware. Each piece can be correct while the combination is not: refresh tokens that can be replayed, logout that does not take effect on other server instances, tokens that can be copied from browser storage and used from another machine, and authorization checks forgotten on a route. The motivation for Ninsho is to provide these pieces as one tested library whose documentation names, for every security property it claims, the automated test that demonstrates it.

A second motivation is evidential. Passing tests are easy to over-read. Over the course of the project, a follow-up review reproduced five defects despite a passing suite, including a race between session creation and revocation. This report therefore treats the tests as scoped evidence, separates what was executed from what was only documented, and states limitations explicitly.

1.3 Problem Statement

Applications that issue session credentials need a single component that keeps credential lifecycle consistent across server instances and browser tabs, detects stolen refresh tokens, optionally prevents replay of stolen access tokens, and separates authentication from authorization, while failing safely when its backing store is unavailable. Existing practice either leaves these concerns to ad-hoc combinations of libraries or documents security

properties without a traceable test for each. This project designs, implements and evaluates an SDK, Ninsho, that provides these behaviours and reports the evidence for and against each property.

1.4 Objectives

The objectives of this project are:

1. To study standards and research on token-based authentication, session revocation, refresh-token security, proof-of-possession and WebAuthn.
2. To design and implement a Node.js SDK that provides opaque or PASETO access tokens, rotating refresh tokens with reuse detection, optional DPoP binding, authorization guards and an injected store with a Redis implementation.
3. To provide a browser client that holds a non-extractable key, refreshes tokens automatically and invalidates peer tabs on logout.
4. To evaluate the behaviour of the SDK through automated tests, manual attack procedures, browser regressions, Redis restart, outage and data-loss scenarios, and a demonstration application.
5. To compare, within a clearly stated scope, the behaviour of a deliberately limited stateless JWT baseline with that of Ninsho in the demonstration application.
6. To document limitations and unverified areas so that claims remain proportionate to the evidence.

1.5 Scope of the Work

The work covers the design, the implementation as it stands at repository commit 6f82026 (release v0.1.1), and the evidence recorded in the repository and in public CI. The following are within scope:

- opaque access tokens, session creation, refresh-token rotation, reuse detection and revocation;
- optional PASETO v4.public access tokens with a store-backed deny-list;
- optional DPoP binding with proof validation and replay protection, and the browser client with cross-tab invalidation;

- authorization guards, rate limiting and framework adapters for Express-style, Fastify, Hono and Koa applications;
- the optional WebAuthn ceremony and attestation verifier;
- Redis persistence, outage and data-loss behaviour; and
- the SecureShop demonstration application.

The following are outside the scope and are not claimed: password storage or verification, federated sign-in, a user study or any deployment in production, certification, an independent security audit, formal verification, physical-device interoperability testing for passkeys, and a performance comparison with JWT libraries. Where the repository records measurements, they are reported as historical measurements of the stated version and machine, not as properties of the current code.

1.6 Organization of the Report

The rest of the report is organized as follows. Chapter 2 reviews the related literature and standards. Chapter 3 presents the proposed methodology: the architecture, the algorithms and the evaluation approach. Chapter 4 discusses the results, separating recorded evidence from unexecuted checks. Chapter 5 concludes the report and outlines future scope. Chapter 6 lists the publications and competitions arising from this work.

Chapter 2

Literature Survey

This chapter reviews standards and research relevant to the design decisions in Ninsho. Because authentication libraries implement specifications, the standards are reviewed together with the peer-reviewed work. Peer-reviewed sources were selected for relevance to web authentication, session security and token implementations; the review does not claim to be an exhaustive survey of the field.

2.1 Review of Existing Work

2.1.1 Authentication schemes and web sessions

Bonneau et al. [10] proposed a framework for comparing web authentication schemes against usability, deployability and security benefits, and used it to compare password-replacement proposals. Their central lesson for library design is that schemes involve trade-offs rather than a single dominant choice, which motivates Ninsho's decision to make stronger options, such as DPoP binding and PASETO tokens, optional and to document their costs. Squarcina et al. [11] studied how weaknesses in cookie handling undermine the integrity of web sessions and proposed defences. Ninsho's example application keeps the refresh token in an HTTP-only cookie, so cookie-layer integrity remains the responsibility of the host application and its deployment, a limitation that is stated in Chapter 5.

2.1.2 OAuth, token theft and refresh-token security

Fett, Küsters and Schmitz [12] analysed OAuth 2.0 in a formal model of the web infrastructure, reported previously unknown attacks and proposed fixes. The IETF OAuth Security Best Current Practice [4] consolidates such findings. Among its recommendations for refresh tokens are sender-constraining and rotation with reuse detection, which Ninsho implements as single-use refresh tokens, a short grace window for concurrent tabs, and revocation of the whole token family when a consumed token is replayed after the grace window. The base framework and bearer-token usage are specified in RFC 6749 [13] and RFC 6750 [2].

2.1.3 JWT security and implementation quality

RFC 7519 [3] defines the JWT format, and RFC 8725 [9] collects best current practices for its use, including strict algorithm handling and validation of claims. Yang et al. [8] developed a testing methodology, JWTeemo, and applied it to 43 JWT implementations in ten programming languages; they reported 31 previously unknown vulnerabilities, 20 of which received CVE identifiers, with authentication bypass in Kubernetes and denial of service against Apache James among the demonstrated impacts. This supports a conservative approach to token parsing. Ninsho's default token is an opaque random string with no parsing surface, and its optional PASETO implementation is written to the specification with a fixed algorithm, strict base64url canonicality checks, and the official test vectors [14]. The study concerns implementation bugs; it does not argue that JWTs cannot be used securely, and neither does this report.

2.1.4 Proof-of-possession and sender-constrained tokens

RFC 9449 [5] specifies DPoP, in which the client proves possession of a private key with a signed proof bound to the HTTP method, the target URI, a unique identifier, a time of issue and, for resource requests, a hash of the access token. The specification recommends that servers track proof identifiers to limit replay within the acceptance window. JWK thumbprints used to bind a token to a key follow RFC 7638 [15]. Ninsho's server verifies these elements and claims each proof identifier atomically in the store; its browser client generates a non-extractable WebCrypto key and persists it in IndexedDB.

2.1.5 Passkeys and WebAuthn

The WebAuthn specification [6], which became a W3C Recommendation on 25 August 2026, defines registration and authentication ceremonies, attestation formats and signature counters. It relies on CBOR [16] and on COSE keys. Lassak et al. [17] studied the obstacles that companies face when deploying FIDO2 passwordless authentication; their focus is organizational deployment rather than the implementation of a verifier. Ninsho's optional WebAuthn package verifies ceremonies and returns a principal, leaving credential storage and user records to the application.

2.2 Summary of Literature

Table 2.1 summarizes the literature reviewed. The column headed "Dataset / Scope" records the subject of each work, since most are standards or analytical studies without datasets.

Table 2.1: Summary of literature survey

Sr. No.	Author(s) & Year	Method / Technique	Dataset / Scope	Key Result / Contribution	Limitation
1	Bonneau et al., 2012 [10]	Comparative evaluation framework	Web authentication schemes (analytical)	Structured comparison; shows trade-offs between schemes	Predates widespread passkeys; not about token lifecycles
2	Fett et al., 2016 [12]	Formal analysis in a web infrastructure model	OAuth 2.0 flows (formal model)	Reported previously unknown attacks and proposed fixes	Models protocol flows, not a particular SDK or store
3	Squarcina et al., 2023 [11]	Analysis of cookie-based session integrity	Web session cookies	Attacks on and defences for session integrity	Cookie layer only; token lifecycle out of scope
4	Lassak et al., 2024 [17]	Study of FIDO2 deployment obstacles	Companies deploying passkeys	Identifies deployment obstacles	Organizational view; no implementation evaluation
5	Yang et al., 2026 [8]	Testing methodology (JWTeemo)	43 JWT libraries, 10 languages	31 new vulnerabilities, 20 CVEs	Implementation bugs; no revocation-design comparison
6	IETF, RFC 8725, 2020 [9]	Best current practice	JWT usage	Practices for safe JWT handling	Guidance, not measured evidence
7	IETF, RFC 9449, 2023 [5]	Standards-track specification	Sender-constrained OAuth tokens	DPoP proofs and replay recommendations	Needs per-request signing and key management
8	IETF, RFC 9700, 2025 [4]	Best current practice	OAuth 2.0 deployments	Rotation and sender-constraining guidance	Guidance for OAuth; not an implementation
9	W3C, WebAuthn L3, 2026 [6]	Recommendation	Passkey ceremonies	Registration, assertion and attestation rules	Leaves storage and sessions to the relying party
10	PASETO Standard [14]	Community specification	Signed and encrypted tokens	Alternative to JOSE tokens	Maintained outside the IETF RFC series

2.3 Research Gaps

Within the sources reviewed, the following gaps motivate this project. They are statements about the reviewed material, not claims about all prior work.

- **Integrated, runnable behaviour.** Standards and studies describe rotation, sender-constraining and revocation separately; they do not provide one small, runnable library in which these behave consistently and can be exercised together.

- **Failure behaviour under store loss.** The reviewed sources say little about what happens to revocation when the backing store is restarted without data, rolled back or unavailable. Ninsho documents and tests these cases for two token strategies.
- **Browser lifecycle races.** Guidance concentrates on the server. Stale refresh responses, concurrent key creation across tabs and cross-tab logout are client-side concerns that the reviewed sources do not address directly.
- **Traceability of security claims.** Security documentation commonly lists properties without naming the test that demonstrates each. This project adopts a claim-to-test mapping and applies it to its own documentation, including correcting overstated claims.
- **Honest comparison baselines.** Comparisons of stateless tokens against sessions often use a deliberately weak baseline. This project states the baseline's limits and does not claim that JWT systems cannot implement revocation or sender-constraining.

Chapter 3

Proposed Methodology

This chapter describes the system, the inputs used to exercise it, the algorithms that implement the token lifecycle, the evaluation metrics and the tools. Ninsho is a software library and not a learning system, so there is no training dataset and no classification accuracy, precision, recall or F1-score in this report. Evaluation is by whether specified behaviours occur, expressed as test outcomes and observed HTTP status codes, and by latency and throughput measurements where the repository records them.

3.1 System Architecture

The block diagram of the proposed system is shown in Figure 3.1. Ninsho is a monorepo of four packages. The application sits between an optional browser client and the Ninsho server package, and Ninsho reads and writes all state through a small store interface.

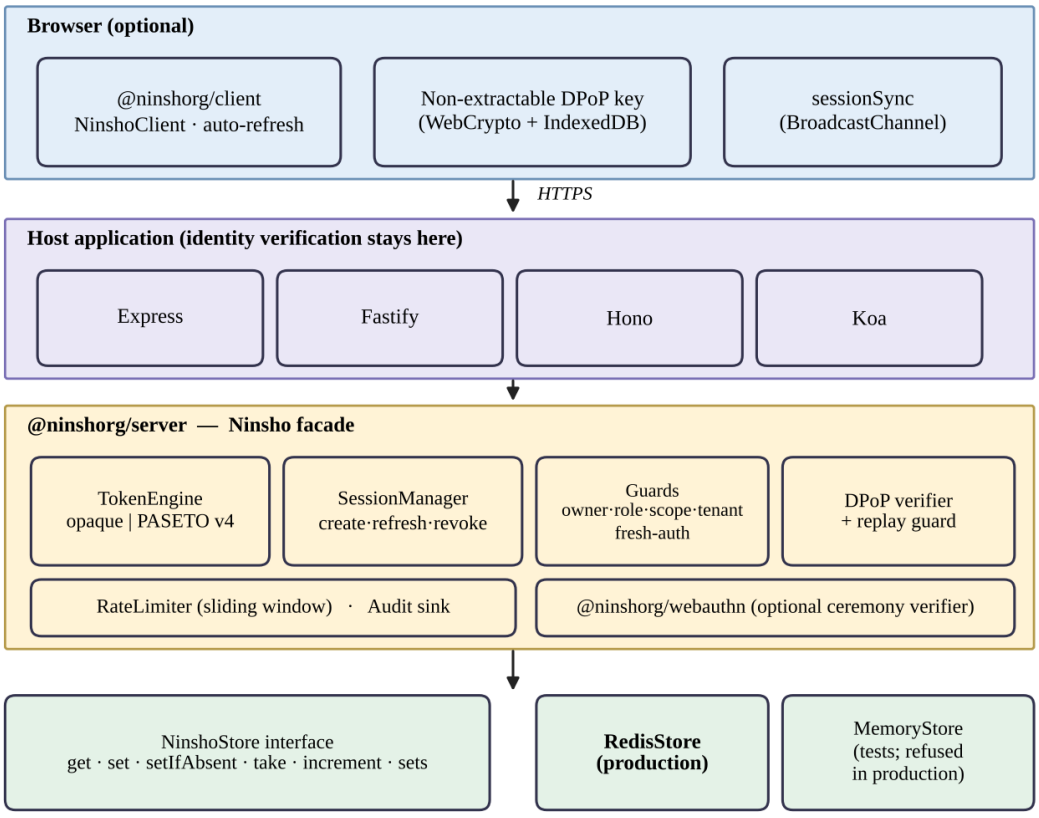


Figure 3.1: Block diagram of the Ninsho architecture

Table 3.1 lists the packages. The server package exposes a facade, `Ninsho`, over a token engine, a session manager, middleware guards, a rate limiter and the DPoP verifier. Each piece is also exported separately, so an application can assemble them differently.

Table 3.1: Packages in the Ninsho monorepo (release 0.1.1)

Package	Role	Runtime dependencies
`@ninshorg/core`	Shared types, errors and cryptographic primitives (token generation, SHA-256 hashing, constant-time comparison)	None
`@ninshorg/server`	Sessions, token engines (opaque and PASETO v4.public), guards, rate limiting, DPoP verification, stores, adapters for Fastify, Hono and Koa	`ioredis`, `@ninshorg/core`
`@ninshorg/client`	Browser client: DPoP key management, automatic refresh, cross-tab invalidation	None
`@ninshorg/webauthn`	WebAuthn registration and authentication verification, attestation formats, test authenticator (separate subpath)	`@ninshorg/core` only

3.1.1 Token engines and the one-read verification path

The default engine issues opaque tokens: 32 bytes from a cryptographically secure generator, encoded in base64url. The server stores only the SHA-256 hash of the token, so a leaked store does not reveal usable tokens. Verification hashes the presented token and performs one store read, as shown in Figure 3.2. Because revocation deletes the record, the same read that authenticates a request also enforces revocation. If the store is unreachable the default policy fails closed with HTTP 503, and for the opaque engine the configuration that would fail open is refused at start-up, because the store is the token.

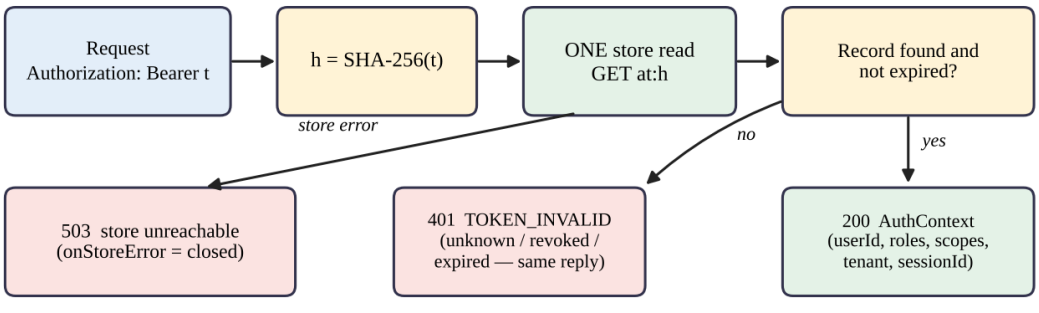


Figure 3.2: Opaque token verification: one store read per request

The optional engine issues PASETO v4.public tokens, which are signed with Ed25519 [18] and verified locally with a key identified by the token footer [14]. Signature verification

needs no store read, but revocation requires a deny-list in the store, and this difference drives the trade-offs summarized in Table 3.2.

Table 3.2: Opaque and PASETO v4.public strategies compared (from repository documentation)

Aspect	Opaque (default)	PASETO v4.public (optional)
Token content	Random lookup key; claims stay on the server	Signed claims readable by the holder
Verification	SHA-256 and one store read	Ed25519 signature check, plus deny-list lookup if revocation is enforced
Revocation	Delete the record; immediate	Deny-list entry in the store until expiry
Store outage, fail-closed	Rejects (503)	Rejects (503)
Store outage, fail-open	Refused at configuration time	Accepts valid signatures without a deny-list check; revoked tokens work during the outage
Complete store data loss	All tokens unknown; users must sign in again	Active tokens keep verifying; previously revoked tokens verify again until expiry

3.1.2 Session manager, refresh tokens and the store

The session manager creates a session, rotates refresh tokens and revokes sessions. A session has a fixed absolute lifetime set at creation, which rotation cannot extend. Defaults are an access-token lifetime of 300 seconds, a refresh-token lifetime of seven days, a refresh grace window of 30 seconds and a clock tolerance of five seconds. All state is read and written through the `NinshoStore` interface, whose operations include `get`, `set`, `setIfAbsent`, an atomic `take` (read and delete), `increment` and set operations. The production implementation, `RedisStore`, requires Redis 6.2 or later because `GETDEL` makes `take` atomic; commands issued during an outage are rejected rather than queued. `MemoryStore` is intended for tests and refuses to be constructed when `NODE_ENV` is `production`.

3.1.3 Authorization guards and framework integration

Authentication answers who the caller is; authorization answers what the caller may do. After the verification middleware has populated the request context, six guards can be mounted: `requireRole`, `requireAllRoles`, `requireScope`, `requireOwner`, `requireTenant` and `requireFreshAuth`. A guard refuses rather than guesses when it cannot decide; for example, an ownership selector that resolves to nothing yields a 403 and not a pass. This addresses the broken object-level authorization risk [7], but the application must still supply

a trustworthy owner for each resource. Adapters for Fastify, Hono and Koa are published as separate subpath exports with no runtime framework dependency.

3.1.4 Browser client and cross-tab invalidation

The browser client keeps the access token in memory, generates a non-extractable WebCrypto key and stores it in IndexedDB, refreshes automatically and attaches DPoP proofs when binding is enabled. Key creation uses a single read-write IndexedDB transaction so that tabs initializing concurrently converge on one key. An opt-in `sessionSync` feature broadcasts credential-free notices over the browser's `BroadcastChannel`, as in Figure 3.3. Notices carry only a version, a context name, an event type and a random identifier; they invalidate peer tabs and never transfer a token. Delivery is best-effort and the server remains authoritative.

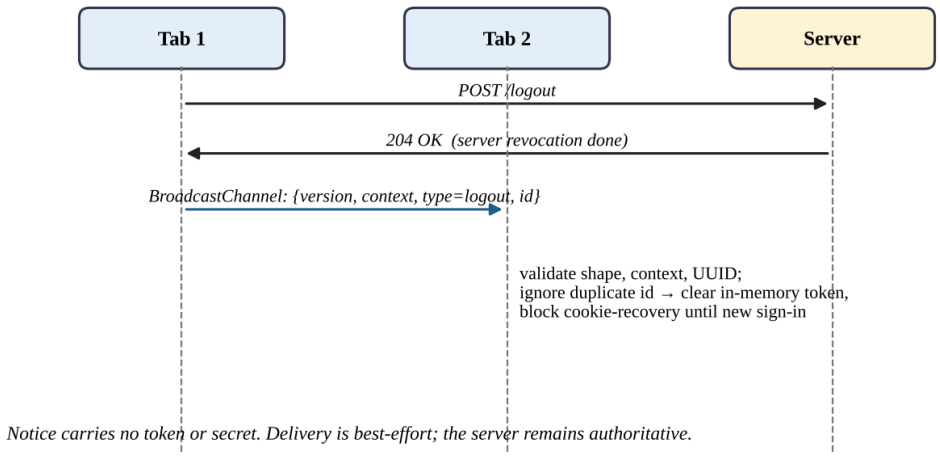


Figure 3.3: Cross-tab invalidation after a confirmed logout

3.1.5 WebAuthn verification

The optional `@ninshorg/webauthn` package verifies the registration and authentication ceremonies: the challenge, the origin, the relying-party identifier, the signature and the signature counter. It parses CBOR with definite lengths only, handles COSE keys, and verifies the attestation formats `none`, `packed`, `apple`, `tpm`, `fido-u2f`, `android-key` and `android-safetynet`, with an optional metadata-service module. It stores no credentials; it returns a principal to the application. A virtual authenticator for tests is exported from a separate subpath and refuses to run under `NODE_ENV=production`.

3.2 Test Data and Experimental Inputs

Ninsho is evaluated with constructed inputs and not with a dataset. Table 3.3 lists the kinds of input used. None of the inputs involves real users or real personal data.

Table 3.3: Test data and experimental inputs

Parameter	Details
Nature of data	Synthetic principals, tokens, DPoP proofs, WebAuthn ceremonies and HTTP requests; no learning dataset
Specification vectors	Official PASETO v4.public vectors 4-S-1 to 4-S-3; the RFC 7638 thumbprint vector; RFC 8949 Appendix A CBOR vectors (as named in repository tests)
Adversarial inputs	Tampered, truncated and malformed tokens; a PASETO review test with 10,000 malformed or adversarial inputs and single-byte mutations of every signature byte; fuzz tests of attacker-controlled parsers
Authenticator fixtures	A virtual authenticator and generated certificates for attestation formats
Application fixtures	Express example and playground applications; manual attack scripts MT-01 to MT-20; SecureShop with two seeded demonstration accounts (an administrator and a customer)
Backing stores	`MemoryStore`; real Redis 7 instances, including disposable Docker instances for persistence tests

3.3 Algorithm Description

The algorithms below summarize the behaviour implemented in `packages/server/src/session/manager.ts`, the token engines and the DPoP module, as read at commit 6f82026. They are descriptions of the implementation and not new algorithms.

3.3.1 Session creation and request verification

1. The application authenticates the user by its own means and calls session creation with a principal.
2. The manager generates a session identifier and a refresh token, stores the refresh record under the token’s SHA-256 hash, and fixes the session’s absolute expiry.
3. It writes session metadata and adds the session to the user’s session index.
4. The engine issues an access token and stores its record under the hash of the token (see Equation (3.1)).
5. The manager checks whether a revocation marker for the session already exists. If so, it revokes the access token and the family and raises an error, so that a credential issued after a completed revocation is never returned.

6. On each request, verification hashes the presented token, reads the record once, checks expiry and DPoP binding, and returns the request context or a 401 or 503 response.

$$h = \text{SHA-256}(t), \quad t = 32 \text{ random bytes from a CSPRNG, base64url-encoded} \tag{3.1}$$

3.3.2 Refresh-token rotation and reuse detection

1. The client presents a refresh token. The manager peeks at its record and, if the token is DPoP-bound, rejects a missing or different key without consuming the token, so that a thief holding only the token cannot destroy the legitimate session.
2. The manager atomically takes the live record. Exactly one concurrent caller wins.
3. The winner writes a replacement token with the next generation number, a grace mapping from the old hash to the new token for 30 seconds, and a tombstone for the old hash, then issues a new access token. After each step it checks for a revocation marker and undoes its writes if one has appeared.
4. A caller that finds no live record looks for the tombstone. If it is found and a grace mapping exists, the caller adopts the same replacement token, so that parallel tabs converge on one chain.
5. If a tombstone exists but the grace window has passed, the token has been replayed: the whole family is revoked, a reuse-detected audit event is emitted, and a reuse error is returned.
6. If neither record exists, the token is rejected as invalid without revealing why.

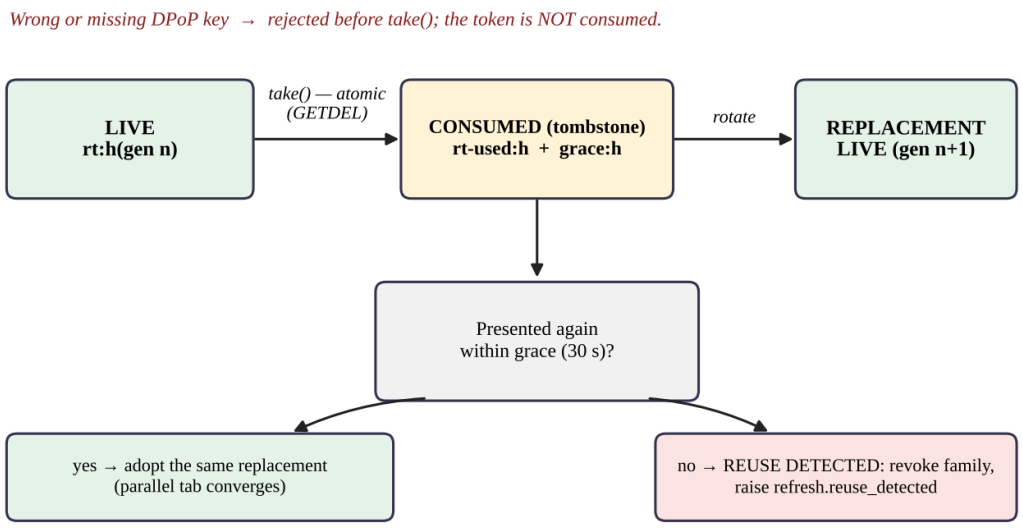


Figure 3.4: Refresh-token states, grace window and reuse detection

3.3.3 Revocation

1. Write a session-revoked marker first, so that concurrent issuance paths can see it.
2. Mark and delete every refresh token, tombstone and grace record in the session's family.
3. Ask the engine to revoke every access token of the session.
4. Remove the session from the user's index. For logout-all, work from one snapshot of the user's indexed sessions, retain any session whose revocation failed, and raise an incomplete-revocation error so the caller can retry.

3.3.4 DPoP proof verification

Figure 3.5 shows the checks that a DPoP-bound request passes, in order. The acceptance window is expressed in Equation (3.2), where the proof maximum age is 60 seconds and the clock tolerance is five seconds by default.

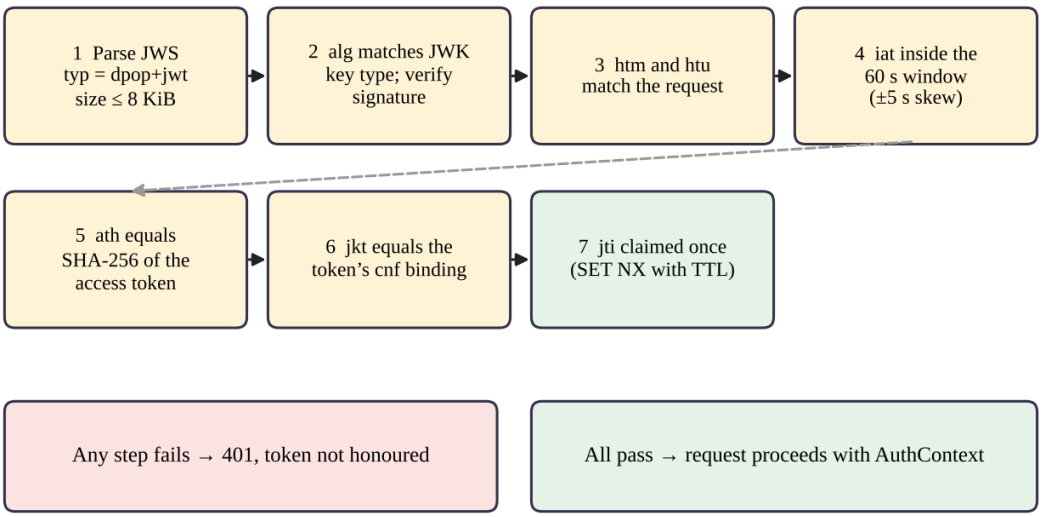


Figure 3.5: DPoP proof validation pipeline

$$\text{now} - (A + \delta) \;\le\; \text{iat} \;\le\; \text{now} + \delta, \quad A = 60\ \text{s}, \quad \delta = 5\ \text{s} \tag{3.2}$$

Proof identifiers are claimed with an atomic set-if-absent operation, keyed by key thumbprint and identifier, with a retention at least as long as the acceptance window, so that two simultaneous replays cannot both succeed. Under an explicit fail-open configuration the replay check is skipped during a store outage; the signature, method, URI and age checks still apply.

3.3.5 Evaluation Metrics

Because outcomes are mostly binary, the principal metrics are counts. For a set of defined scenarios, conformance is the fraction of applicable scenarios whose observed outcome equals the expected outcome (see Equation (3.3)). It is reported with the counts and is not an accuracy figure for a classifier. For timing measurements recorded in the repository, throughput is operations per second over a run (see Equation (3.4)), reported with mean, 95th and 99th percentile latency.

$$\text{Conformance} = \text{scenarios passed} \div (\text{scenarios run} - \text{not applicable}) \tag{3.3}$$

$$\text{Throughput} = \text{operations completed} \div \text{run time in seconds} \tag{3.4}$$

Further reported quantities are the number of automated tests and skipped tests, the outcome of each browser engine, and the outcome of each Redis scenario. No precision, recall or accuracy measure is reported because no classification is performed.

3.4 Tools and Technologies Used

- **Programming language:** TypeScript on Node.js 20 or later (tests recorded on Node 20, 22 and 24).
- **Libraries and frameworks:** `ioredis` for Redis; Node's built-in `node:crypto`; WebCrypto, IndexedDB and `BroadcastChannel` in the browser; Express, Fastify, Hono and Koa for adapters and examples; `jose` in the SecureShop JWT baseline.
- **Build and test:** `tsup` for dual ESM and CommonJS builds, Vitest for tests, `fake-indexeddb` for client key-store tests, Playwright 1.64.1 for the Chromium, Firefox and WebKit regressions.
- **Infrastructure:** Redis 7 (Docker images `redis:7-alpine`), npm workspaces, GitHub Actions.
- **Hardware and operating systems: the fresh re-run of 21 October 2026 used a Dell Latitude E5440 (Intel Core i5-4300U, 2 cores / 4 threads, 16 GB RAM) running Windows 10 Home (build 19045) with Redis 7.4.11 in Docker Desktop (WSL2); CI runs on Ubuntu.**

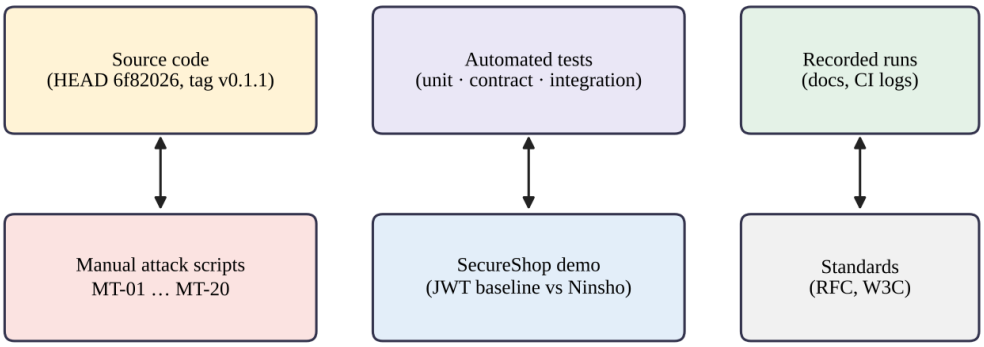


Figure 3.6: Evidence sources used in this report

Chapter 4

Results and Discussion

This chapter reports only what the repository, public CI records and the project's own documents support. Each result is labelled with who produced it and whether it was independently re-executed while this report was prepared. Passing tests are evidence that specified behaviours held under the tested conditions. They do not demonstrate that all attacks are prevented and they do not constitute an independent security audit.

4.1 Experimental Setup

Table 4.1 classifies the evidence used. "Project-recorded" means the result is documented in the repository as a run by the project's authors. "CI-confirmed" means the result was read from GitHub's public API for the project's Actions run on the commit examined. "Re-run" means the result was reproduced on 21 October 2026 on the machine in Table 4.2; its logs, probes and benchmark files are in the repository under research/validation.

Table 4.1: Evidence categories and their status

Evidence	Source	Status
Full test suite with real Redis	Repository documents (review remediation, cross-tab guide, browser guide)	Project-recorded; not re-run for this report
CI on commit 6f82026	GitHub Actions run 38607481028 (push to main, 14 Oct 2026), all 8 jobs succeeded	CI-confirmed (read on 20 Oct 2026)
Browser regressions (3 engines)	Local results in the browser guide; CI matrix of three jobs	Project-recorded and CI-confirmed
Manual attack procedures MT-01 to MT-20	`manualtest/TEST-RESULTS.md` and evidence files, written against release 0.1.0	Project-recorded; not re-run on 0.1.1
Redis persistence scenarios	`redis-persistence.test.ts` and operations guide	Project-recorded; CI execution of the Docker-managed suite not established
SecureShop outcomes	SecureShop README and test file	Documented; execution log [to be attached]
Benchmarks	Performance documents for release 0.1.0	Historical; not for the current commit
Fresh re-run by the authors	21 Oct 2026; commit 6f82026 plus one local fix (F-01); Node 22.13.1, npm 11.0.0, Redis 7.4.11; logs in research/validation/logs	Re-run: build, typecheck, 2,151 tests, 6 browser runs, 7 Redis persistence scenarios, SecureShop 3/3

Table 4.2: Experimental environments recorded in the repository

Item	Recorded value
Commit examined	6f82026 (feat(demo): add SecureShop viva showcase application); packages published as 0.1.1 from commit f784566 with identical package source
Local runs (project-recorded)	Windows; Node 22.13.1 (browser and cross-tab runs) and 24.19.0 (viva rehearsal); npm 11.0.0; dedicated Redis 7.4.8 and 7.4.11 in Docker; Playwright 1.64.1
CI	GitHub Actions: Ubuntu runners; Verify on Node 20 and 22 with a `redis:7-alpine` service; browser jobs on Ubuntu 24.04 with Node 22; lockfile, dependency audit and bundle-purity jobs
Local hardware	Fresh re-run: Dell Latitude E5440, Intel Core i5-4300U (2 cores / 4 threads), 16 GB RAM, Windows 10 Home 10.0.19045, Docker Desktop 4.38.0

4.2 Results

4.2.1 Automated test suite and continuous integration

The project recorded a full run (build, type-check, tests) against a dedicated Redis instance with no skipped tests. The browser guide and the cross-tab guide record 2,142 workspace tests in 53 files plus one HTTP demonstration test; the review-remediation document records 2,116 tests with no skips on Node 20, 22 and 24.19.0. The two figures describe different working-tree states of the project and neither is a count for the final commit, so this report does not present either as the count for commit 6f82026. Figure 4.1 shows the per-package breakdown recorded with the 2,142 figure.

A fresh run on 21 October 2026 against a dedicated Redis 7.4.11 instance, on commit 6f82026, passed 2,150 tests with none skipped (client 117, core 97, server 1,116, webauthn 623, Express example 101, playground 95, HTTP demonstration 1); all seven Docker-managed Redis persistence scenarios executed. After adding one regression test for defect F-01 the count is 2,151. In the first post-fix run two Docker persistence tests failed while starting a container (a container-name conflict while Docker was still removing the previous one); the server package was re-run immediately and all 1,117 of its tests passed. This is reported as an intermittent test-harness failure, not hidden.

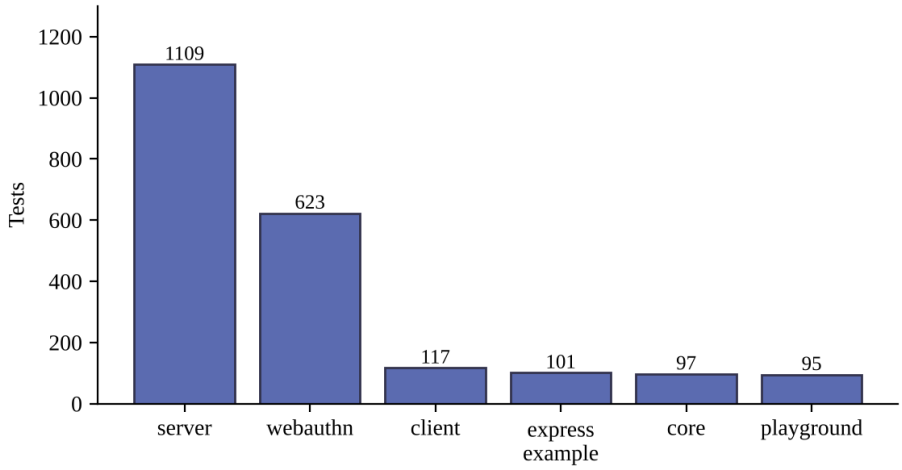


Figure 4.1: Tests per workspace in the recorded run (project-recorded)

On the commit examined, all eight jobs of the GitHub Actions run succeeded: Verify on Node 20 and Node 22 (with a Redis service), Chromium, Firefox and WebKit browser jobs, the lockfile-consistency job, the dependency audit of production dependencies, and the bundle-purity job, which also installs the packed tarballs as a consumer would and loads them as ESM and CommonJS. The single skipped step was the benchmark smoke run on Node 20, which is skipped by design. One caveat applies: the CI step that checks that the Redis persistence suite ran searches the output for the suite name, and the placeholder printed when Docker is unavailable has the same name, so that step alone cannot show the Docker-managed persistence tests executed on the runner.

4.2.2 Security behaviours and their test evidence

Table 4.3 maps the principal security behaviours to the test files that the repository's documentation names for them. The mapping was checked against the file list at the commit examined; passing status derives from the successful CI run above and from the project-recorded runs.

Table 4.3: Security behaviours and the test files that evidence them

Behaviour	Primary test evidence (file in repository)	Status
Refresh rotation, single winner, reuse detection and grace window	`session.test.ts`, `store-invariants.test.ts`	Passing (CI)
Revocation racing initial issuance; sender-binding checked before consumed-token reuse handling	`review-regressions.test.ts`	Passing (CI)
DPoP: stolen token useless without key; refresh tokens bound	`dpop-integration.test.ts`	Passing (CI)
DPoP proof parsing and thumbprint against the RFC 7638 vector	`dpop-proof.test.ts`	Passing (CI)
DPoP replay rejected; method and URI mismatch rejected	`dpop-request.test.ts`, `dpop-integration.test.ts`	Passing (CI)

Behaviour	Primary test evidence (file in repository)	Status
Ownership, role, scope, tenant and fresh-authentication guards	`middleware.test.ts`, `fresh-auth.test.ts`	Passing (CI)
PASETO v4.public official vectors and tamper resistance	`paseto-v4.test.ts`, `paseto-independent-audit.test.ts`	Passing (CI)
Store contract holds for MemoryStore and RedisStore; fail-closed against an unreachable Redis	`store.contract.test.ts`, `store-invariants.test.ts`	Passing (CI, Redis job)
Rate limiting without boundary double-burst; fail-closed on store error	`ratelimit.test.ts`	Passing (CI)
Browser key store, lifecycle races and cross-tab sync	`keys.test.ts`, `storage.test.ts`, `lifecycle.test.ts`, `session-sync.test.ts`	Passing (CI)
WebAuthn ceremonies, attestation formats, CBOR and COSE parsing	`ceremony.test.ts`, `attestation.test.ts`, `cbor.test.ts`, `cose.test.ts`	Passing (CI)

The repository also contains an in-repository cryptographic review of the PASETO implementation. It adds an independent reference implementation of the pre-authentication encoding and 44 tests, including exhaustive single-byte mutations of the signature. This was carried out within the project and is not an external audit, even though it uses an implementation independent of the library's own helpers.

4.2.3 Manual attack procedures

Twenty manual procedures, with five follow-up retests, were recorded against the example applications. They give 25 recorded entries: 23 passed and 2 were marked not applicable because no resource endpoint of the required kind existed in the shipped applications. By Equation (3.3), the conformance over applicable entries was 23 of 23. These procedures were recorded against release 0.1.0 and have not been re-run on 0.1.1. Table 4.4 groups them.

Table 4.4: Manual attack procedures grouped by theme (project-recorded, release 0.1.0)

Theme	Entries	Recorded outcome
Token tampering and signature, footer, issuer and audience attacks	MT-01 to MT-03, MT-01B, MT-19, MT-19B	Rejected with 401
Replay: revoked token, expired token	MT-04, MT-05, MT-05B, MT-06	401 after logout and after expiry
Refresh replay and rotation abuse	MT-07, MT-08	Reuse after the grace window rejected and family revoked; concurrent refresh converged on one replacement token
Authorization: cross-user, role, scope, tenant	MT-09 to MT-12	403 for other users' resources and for missing role; MT-11B and MT-12B not applicable
Rate limiting	MT-13	Fifth failed attempt allowed; sixth returned 429 with a retry hint
DPoP: missing proof, method, URI and replay	MT-14 to MT-17	Rejected in each case
Step-up authentication and malformed input	MT-18, MT-20	Stale authentication refused; oversized body 413; malformed cookie handled without a crash

4.2.4 Browser evidence

Two regression scripts run unchanged scenarios on Chromium, Firefox and WebKit through Playwright. The key-store script gates two pages' initial IndexedDB loads, creates keys concurrently and checks that both pages receive the same non-extractable key which survives reload. The session-sync script checks that a peer logout invalidates state, that a delayed refresh cannot restore it, that a delayed sign-in cannot override a peer identity change, and that the shared key survives logout. Local results recorded by the project are in Table 4.5; the three browser jobs in the GitHub Actions run on Ubuntu also succeeded. The local and CI results are distinct measurements and are reported separately.

Table 4.5: Browser regression results (project-recorded local run on Windows, Node 22.13.1, Playwright 1.64.1)

Engine	Version reported	Key-store	Session-sync	CI job on 6f82026
Chromium	156.0.8078.4	PASS	PASS	Succeeded
Firefox	157.0	PASS	PASS	Succeeded
WebKit	27.2	PASS	PASS	Succeeded

The scripts use controlled `fetch` stubs for network responses. They do not establish real refresh-cookie ordering, server-side revocation, behaviour of suspended or background tabs, or interoperability on physical devices, and Playwright's WebKit is an automation build rather than shipped Safari.

4.2.5 Redis persistence, outage and data loss

Disposable Redis 7 instances managed through Docker were used to test four scenarios for both token strategies. Table 4.6 summarizes the recorded behaviour. For the opaque strategy every scenario fails closed. For the PASETO strategy, loss or rollback of Redis state is a documented, unresolved limitation: revoked tokens verify again until they expire. The persistence behaviour itself follows the underlying Redis modes; an RDB snapshot can lose recent writes after a crash, and the append-only file narrows but does not remove that window [19]. Abrupt crash, append-only durability, Sentinel and cluster failover were not tested by the project.

Table 4.6: Redis scenarios and recorded outcomes (project-recorded; Docker-managed tests)

Scenario	Opaque strategy	PASETO strategy
Graceful restart with RDB	Sessions and revocation state preserved	Sessions and deny-list preserved
Temporary outage, fail-closed (default)	All verification rejected (503); recovers when Redis returns	Rejected (503); recovers
Complete data loss (empty restart)	All tokens unknown; users must re-authenticate	Active tokens still verify; previously revoked tokens verify again until expiry
Snapshot rollback	Tokens created after the snapshot fail closed	Tokens revoked after the snapshot are resurrected
Outage with fail-open configured	Refused at configuration time	Valid signatures accepted without deny-list check; audit event emitted

4.2.6 SecureShop demonstration

SecureShop is a demonstration shop application that installs the published `@ninshorg/server` and `@ninshorg/client` 0.1.1 packages and runs three authentication paradigms under the same application requirements. Comparison A uses a stateless HS256 JWT issued with the `jose` library and stored in browser `localStorage`, with a five-minute expiry and no server-side state. Comparison B uses Ninsho's opaque tokens on two application instances that share a Redis store. Comparison C uses Ninsho with DPoP binding, an in-memory access token, a non-extractable key in IndexedDB, an HTTP-only refresh cookie and cross-tab synchronization. The Security Experiments Lab in the application issues the replay requests so an examiner can see each response. Two accounts are seeded, an administrator and a customer, with demonstration credentials that must never be used in a real system.



Figure 4.2: SecureShop, Comparison A: replay of a copied JWT after client logout [insert screenshot]



Figure 4.3: SecureShop, Comparison C: wrong-key and replayed-proof requests [insert screenshot]

The outcomes documented for SecureShop are given in Table 4.7. The repository records an executed rehearsal for the related three-demo script, in which all ten expected statuses (200, 200, 204, 401, 401, 401, 200, 401, 200, 403) matched on Node 24.19.0 with Redis 7.4.11. SecureShop's own automated tests were re-run on 21 October 2026 (Node 22.13.1) and passed 3 of 3.

Table 4.7: SecureShop documented outcomes by comparison

Request	A: stateless JWT baseline	B: Ninsho opaque	C: Ninsho with DPoP
Copied token replayed after client logout	HTTP 200 (accepted until expiry)	HTTP 401 on both instances after server revocation	Server revokes the session
Copied token replayed before logout	Accepted	Accepted (bearer copy works by design)	401 without a proof
Proof signed by a different key	Not applicable	Not applicable	401
Replay of an already used proof	Not applicable	Not applicable	401
Peer tab after logout	No mechanism	No mechanism	Notified through BroadcastChannel

Fresh run, 21 Oct 2026: npx vitest run in demo/viva-showcase — 1 file, 3 tests passed (Node 22.13.1); log research/validation/logs/secureshop-test.log.

4.2.7 Historical benchmark measurements

The repository contains benchmark reports for release 0.1.0, produced on an Intel Core i5-4300U machine with Node 22.13.1, and an earlier table on a different machine. They are historical measurements of library overhead and not figures for the current commit. Figure 4.4 reproduces two recorded throughput comparisons between Ninsho's own strategies: in memory, opaque verification was recorded at 17,267 operations per second against 2,922 for PASETO, while with a Redis round trip the figures were 507 and 418. Refresh rotation was recorded at 7,551 operations per second in memory and 43 over Redis loopback. The relative result depends on the hardware and on whether the store is local, which the performance document itself notes.

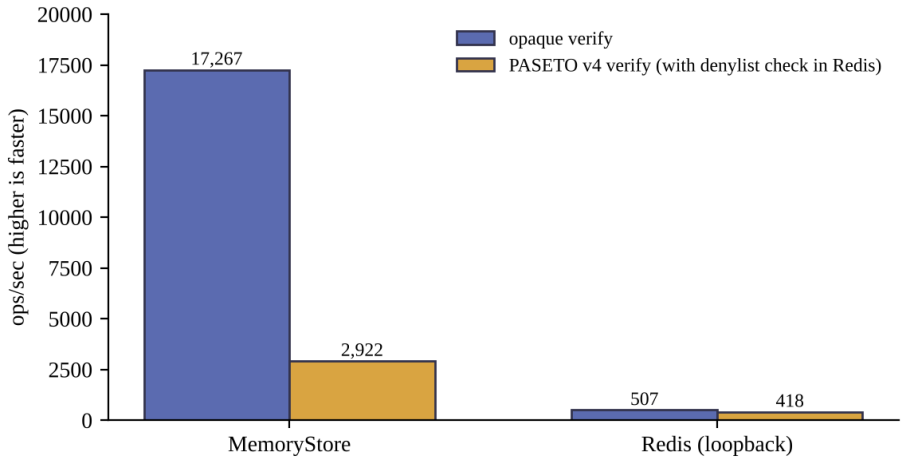


Figure 4.4: Historical verification throughput, release 0.1.0 (not for the current commit)

A fresh, single-machine benchmark was run on 21 October 2026 with the harness in research/validation/benchmarks (method in METHODOLOGY.md; raw samples kept). It used the machine in Table 4.2, Node 22.13.1 and Redis 7.4.11, three independent trials in shuffled order with warm-up, and reports medians. Verification cost at concurrency 1 (median p50): Ninsho opaque with Redis 1.59 ms; Ninsho PASETO with its Redis deny-list check 2.05 ms; and a JWT baseline using the jose library with the same signature algorithm (EdDSA/Ed25519) and an equivalent Redis deny-list check 2.16 ms. One Redis round trip alone took 1.34–1.59 ms, so the store round trip dominated every configuration that provides server-side revocation; signature checking without revocation took about 0.32 ms for both PASETO and JWT.

Over HTTP (one Express 5 application per configuration, identical route and response, closed-loop keep-alive load) the three configurations with server-side revocation reached median throughputs of 287 (opaque), 245 (PASETO) and 251 (JWT with Redis) requests per second at concurrency 1, with p50 latencies of 3.2, 3.7 and 3.7 ms, and 951, 564 and 801 requests per second at concurrency 16 (p50 14.9, 24.9 and 17.6 ms). Before timing, every configuration was checked to reject missing, tampered, revoked and wrong-audience tokens. A stateless JWT reference reached 488 and 963 requests per second but accepted a revoked token until expiry, so it is not a like-for-like comparison. Trial-to-trial variation reached about 30 percent, and the load generator, server and Redis shared two CPU cores alongside other containers; differences inside the reported ranges should not be read as a ranking. These figures describe one machine and one JWT library, not JWT-based systems in general.

4.3 Comparison with Existing Methods

Table 4.8 compares the design of three approaches. The first column is the deliberately limited JWT baseline used in SecureShop, which is a teaching device. A JWT-based system can add a deny-list, a per-user session version, introspection or sender-constrained tokens, and the standards describe how [3][5][9]. The comparison therefore shows what each baseline provides without extra infrastructure; it does not rank JWT and Ninsho in general.

Table 4.8: Comparison of the SecureShop baseline with Ninsho configurations (design properties and documented outcomes)

Property	A: stateless JWT baseline	B: Ninsho opaque	C: Ninsho with DPoP
Server state per token	None	One record per token in the store	One record per token, plus proof identifiers
Revocation before expiry	Not available without added state	Immediate across instances sharing the store	Immediate; peer tabs notified
Use of a copied token elsewhere	Works until expiry	Works until revoked	Rejected without the bound key
Replay of a captured proof	Not applicable	Not applicable	Rejected (identifier claimed once)
Behaviour if the store is lost	Not applicable	Fails closed	Fails closed (opaque); PASETO variant can resurrect revoked tokens
Per-request work	Signature check	One store read	Store read, proof check and an atomic claim; client signs
Protects against malicious script in the page	No	No	No; script in the origin can request signatures

The per-request work row describes design and is not a measurement. The last row is a limitation that the project states explicitly: DPoP with a non-extractable key prevents key export, not use of the key by hostile code running in the same origin.

4.4 Limitations and Threats to Validity

- **No independent audit. All evidence is produced by the project's authors or by automated tooling they wrote. The 21 October 2026 review was model-assisted (an AI coding assistant working with the project's tests) and is not an independent audit. Passing tests are not a security audit.**
- **Test counts are configuration-specific. Earlier documents record 2,116 and 2,142 for other working-tree states; the fresh count for this commit is 2,150, or 2,151 with the F-01 regression.**
- **Single local re-run. The fresh results come from one Windows machine on Node 22; Node 20 and Linux were covered only by the CI run on the examined commit.**
- **Version drift.** The manual attack procedures and benchmarks were recorded against release 0.1.0.
- **Stubbed browser networking, automation browsers and no physical devices.** See Section 4.2.
- **CI coverage of the Docker persistence suite is not established** for the reason given in Section 4.2.1.
- **Demonstration baseline.** The JWT baseline is deliberately limited, and the demonstration accounts and in-memory data are for illustration.
- **Internal validity of the manual tests.** Some attack scripts exercise the example applications and not every deployment configuration.

Chapter 5

Conclusion and Future Scope

5.1 Conclusion

This project designed and evaluated Ninsho, an authentication and session-management SDK for Node.js. The implemented system provides opaque access tokens with single-read verification and immediate revocation, single-use refresh tokens with a grace window and reuse detection, optional PASETO v4.public tokens, optional DPoP proof-of-possession with atomic replay protection, authorization guards, rate limiting, framework adapters, a browser client with a non-extractable key and cross-tab invalidation, and an optional WebAuthn verifier, all over an injected store with a Redis implementation.

The evidence examined supports these behaviours under the tested conditions: the project's own runs and the GitHub Actions run on the examined commit show a passing suite with real Redis and passing Chromium, Firefox and WebKit regressions; recorded manual attack procedures were rejected as expected; and the Redis scenarios document how each strategy fails. Table 5.1 relates the objectives of Section 1.4 to the evidence.

Table 5.1: Objectives and the evidence supporting them

Objective	Status	Evidence and caveat
1. Study standards and research	Met	Chapter 2; sources verified against publisher pages
2. Design and implement the SDK	Met	Repository at commit 6f82026; packages published as 0.1.1
3. Browser client with non-extractable key and cross-tab invalidation	Met, with limits	Browser regressions on three engines; stubbed networking; best-effort delivery
4. Evaluate by tests, attacks, browsers, Redis and demonstration	Met; re-executed locally	Fresh local re-run (21 Oct 2026) of tests, browsers, Redis scenarios and SecureShop; one defect found and fixed (F-01)
5. Compare a limited JWT baseline with Ninsho	Met, narrowly	SecureShop outcomes, plus a single-machine benchmark against a JWT baseline with equivalent Redis revocation (Section 4.2.7)
6. Document limitations	Met	Section 4.4 and the list below

The results are scoped. They do not show that Ninsho is more secure or faster than JWT-based systems in general, and they are not an independent security audit. The following limitations should be read with them:

- Ninsho has not been independently audited, is pre-1.0, and is not certified.

- With the PASETO strategy, loss or rollback of Redis state revives revoked tokens until expiry; with fail-open configuration, revocation and replay checks are skipped during an outage. Opaque mode fails closed.
- Cross-tab notices are best-effort and unauthenticated by design; a failed delivery leaves a peer tab to discover the logout from the server.
- DPoP with a non-extractable key does not defend against malicious script in the same origin.
- Session creation, rotation and revocation are multi-step operations, not one transaction: a failed rotation can consume the old refresh token, and logout-all works on a snapshot of indexed sessions.
- The immediate-login registration route of the example reveals account existence through its response; applications that need enumeration resistance should verify email first.
- Physical-device passkey interoperability, abrupt Redis crash, Sentinel and cluster behaviour, suspended-tab behaviour and real-cookie ordering were not tested.
- There is no DPoP server-nonce mechanism in the implementation reviewed; DPoP replay protection relies on the proof identifier and acceptance window.

5.2 Future Scope

- Commission an independent security review and publish its findings, then re-run all manual procedures against the final release.
- Re-run the full suite, browser matrix and benchmarks on a documented machine, record logs with the commit hash, and publish a reproducibility package.
- Repeat the single-machine performance comparison of Section 4.2.7 on Linux hosts and dedicated hardware, with more trials and confidence intervals.
- Add durability options, such as guidance and tests for append-only persistence, Sentinel and cluster failover, and consider a signed or replicated deny-list for the PASETO strategy.
- Evaluate usability with developers unfamiliar with the library, measuring time to integrate and configuration errors.
- Extend the passkey work with a physical-authenticator interoperability matrix.

- Investigate a server-nonce option for DPoP and an authenticated channel for cross-tab notices.

Chapter 6

List of Publications

6.1 List of Published Papers

None at the time of writing. [Students to confirm; add any paper or preprint in IEEE format, with venue, volume, pages, year and DOI.] The npm releases of the software packages are software releases and are not counted as publications.

6.2 Details of Participation in Project Competitions

None at the time of writing. [Students to confirm; add the competition name, organizer, date and outcome if applicable.]

References

[1] Ninsho project, source repository, commit 6f82026 (release v0.1.1), 2026. [Online]. Available: https://github.com/NinshoORG/ninsho

[2] M. Jones and D. Hardt, “The OAuth 2.0 Authorization Framework: Bearer Token Usage,” RFC 6750, IETF, Oct. 2012, doi: 10.17487/RFC6750.

[3] M. Jones, J. Bradley, and N. Sakimura, “JSON Web Token (JWT),” RFC 7519, IETF, May 2015, doi: 10.17487/RFC7519.

[4] T. Lodderstedt, J. Bradley, A. Labunets, and D. Fett, “Best Current Practice for OAuth 2.0 Security,” RFC 9700 (BCP 240), IETF, Jan. 2025, doi: 10.17487/RFC9700.

[5] D. Fett, B. Campbell, J. Bradley, T. Lodderstedt, M. Jones, and D. Waite, “OAuth 2.0 Demonstrating Proof of Possession (DPoP),” RFC 9449, IETF, Sep. 2023, doi: 10.17487/RFC9449.

[6] W3C, “Web Authentication: An API for accessing Public Key Credentials – Level 3,” W3C Recommendation, Aug. 25, 2026. [Online]. Available: https://www.w3.org/TR/webauthn-3/

[7] OWASP Foundation, “OWASP Top 10 API Security Risks – 2023.” [Online]. Available: https://api-security.owasp.org/editions/2023/en/0x11-t10/

[8] J. Yang, E. Wang, J. Chen, Q. Wang, Y. Zhang, H. Duan, W. Xie, and B. Wang, “Token time bomb: Evaluating JWT implementations for vulnerability discovery,” in Proc. Network and Distributed System Security Symp. (NDSS), 2026. [Online]. Available: https://www.ndss-symposium.org/ndss-paper/token-time-bomb-evaluating-jwt-implementations-for-vulnerability-discovery/

[9] Y. Sheffer, D. Hardt, and M. Jones, “JSON Web Token Best Current Practices,” RFC 8725 (BCP 225), IETF, Feb. 2020, doi: 10.17487/RFC8725.

[10] J. Bonneau, C. Herley, P. C. van Oorschot, and F. Stajano, “The quest to replace passwords: A framework for comparative evaluation of web authentication schemes,” in Proc. IEEE Symp. Security and Privacy (S&P), 2012.

[11] M. Squarcina, P. Adão, L. Veronese, and M. Maffei, “Cookie crumbles: Breaking and fixing web session integrity,” in Proc. 32nd USENIX Security Symp., Anaheim, CA, USA, Aug. 2023, pp. 5539–5556.

[12] D. Fett, R. Küsters, and G. Schmitz, “A comprehensive formal security analysis of OAuth 2.0,” in Proc. ACM SIGSAC Conf. Computer and Communications Security (CCS), 2016; arXiv:1601.01229.

[13] D. Hardt, Ed., “The OAuth 2.0 Authorization Framework,” RFC 6749, IETF, Oct. 2012, doi: 10.17487/RFC6749.

[14] PASETO Standard, “PASETO: Platform-Agnostic SEcurity TOkens – specification,” GitHub repository. [Online]. Available: https://github.com/paseto-standard/paseto-spec

[15] M. Jones and N. Sakimura, “JSON Web Key (JWK) Thumbprint,” RFC 7638, IETF, Sep. 2015, doi: 10.17487/RFC7638.

[16] C. Bormann and P. Hoffman, “Concise Binary Object Representation (CBOR),” RFC 8949 (STD 94), IETF, Dec. 2020, doi: 10.17487/RFC8949.

[17] L. Lassak, E. Pan, B. Ur, and M. Golla, “Why aren’t we using passkeys? Obstacles companies face deploying FIDO2 passwordless authentication,” in Proc. 33rd USENIX Security Symp., 2024, pp. 7231–7248.

[18] S. Josefsson and I. Liusvaara, “Edwards-Curve Digital Signature Algorithm (EdDSA),” RFC 8032 (Informational), IETF, Jan. 2017, doi: 10.17487/RFC8032.

[19] Redis Ltd., “Redis persistence,” Redis documentation. [Online]. Available: https://redis.io/docs/latest/operate/oss_and_stack/management/persistence/